LoopProof

Evidence-Gated E-Waste Traceability

> *"Collected does not mean recycled. Environmental impact must not be claimed until the required evidence exists."*
>
> An evidence-gated traceability framework enforcing role separation, SHA-256 evidence-file integrity checking, and mass-balance conservation before any sustainability claim is recognized.

LIVE INTERACTIVE PROTOTYPE
loop-proof.vercel.app

PUBLIC REPOSITORY
github.com/vishu2212/LoopProof

COMPETITION TRACK
Sustainable Supply Chains (Primary Track)

HACKATHON WINDOW
October 5, 2026 – October 25, 2026

IMPLEMENTATION STATUS
Prototype with 162 Solidity tests • 55 web tests passing

TECHNOLOGY STACK
Next.js 16, TypeScript, Supabase Auth, PostgreSQL RLS, Solidity 0.8.28 (Hardhat)

Project Motivation & Architecture Overview

Global electronic waste generation exceeds 62 million metric tons annually, yet documented formal recycling rates hover below 23%. In extended producer responsibility (EPR) reporting and corporate sustainability disclosures, an institutional failure mode routinely recurs: **collection is conflated with verified processing**. Drop-off receipts and aggregation logs are booked as proof of material recovery, enabling premature carbon offsets and ESG claims while equipment is in transit, stockpiled, downcycled, or lost to informal overseas processing.

LoopProof addresses this verification gap by treating e-waste tracking as a strict, evidence-gated finite state machine. Rather than accepting vendor self-certifications or claiming that blockchain magically guarantees physical recycling, LoopProof enforces cryptographic accountability: each lifecycle transition requires assigned role authorization, non-reusable SHA-256 evidence-file integrity checking, and physical mass-balance validation (total reported output mass ≤ recorded intake mass).

Crucially, numerical environmental impact metrics remain completely unavailable until an independent verifier accepts the claim. The live application demonstrates this workflow using Supabase Auth and PostgreSQL Row Level Security, alongside a reference smart contract suite tested locally on Hardhat.

Core Architectural Pillars

01 / INTEGRITY

Evidence-File Integrity Checking

No lifecycle stage advances on assertion alone. Every state transition requires anchoring an unalterable SHA-256 digest of primary source documentation (intake manifests, facility weigh-in tickets, or refining reports). A digest confirms file non-alteration post-submission, while physical auditing is delegated to an accredited third party.

`Rule D-04 • Global Hash Uniqueness`

02 / GOVERNANCE

Separation of Duties & Role Locks

Rigid separation between Registrar, Collector, Processor, and Verifier. Assignees are immutably bound at batch registration. The independent verifier must be different from the assigned collector and processor; self-verification is programmatically rejected at the database and contract layer.

`Rule D-15 • Enforced Independence`

03 / CONSERVATION

Mass-Balance & Impact Gating

Physical mass conservation is enforced at integer gram precision: total reported output mass ≤ recorded intake mass across all output material fractions. Numerical environmental impact estimates are cryptographically locked and unavailable until verification is complete.

`Rule D-05 • Conservation Law`

RIGOROUS VERIFICATION SUMMARY

162 passing Solidity Mocha tests on local Hardhat EVM • 55 passing web & schema guard tests • 0 typecheck or build errors • Zero PII on-chain policy.

PROTOTYPE ACCESS & CODEBASE

Interactive App: loop-proof.vercel.app
Public Code: github.com/vishu2212/LoopProof

1. The E-Waste Verification Gap

CONTEXT & FAILURE MODES

The Conflation of Intake with Outcome

In conventional e-waste supply chains, certificates of recycling are commonly issued upon initial drop-off at municipal centers or corporate buyback facilities. Aggregators record intake weight, issue a collection manifest, and enterprise customers immediately calculate avoided emissions in corporate ESG disclosures.

This creates a dangerous moral hazard. Between collection and genuine mechanical or chemical refining lies a complex, multi-tiered broker network. Batches are frequently stockpiled, abandoned during commodity price drops, shredded without toxic fraction capture, or exported to developing regions under the guise of "second-hand electronics." Collection demonstrates logistics; it does not demonstrate recycling.

The Oracle Problem in Physical Supply Chains

Many enterprise blockchain proposals claim to solve supply chain opacity by stamping receipts onto a ledger. However, they frequently succumb to the *supply chain oracle problem*: asserting that recording data on a distributed ledger somehow guarantees physical truth in the real world.

A blockchain ledger cannot inspect a chemical leaching bath, verify furnace operating temperatures, or detect counterfeit lab assay sheets. Claiming that "blockchain proves recycling happened" is factually false and undermines engineering integrity. Software can only guarantee what was recorded, by whom, at what time, and whether the data obeyed immutable physical and role-based constraints.

2. The LoopProof Approach: Cryptographic Accountability

DESIGN PRINCIPLES

LoopProof departs from speculative automation. Instead of asserting that software guarantees physical reality, LoopProof models **evidence-anchored accountability**. We define exactly what the ledger proves—and explicitly disclose what it does not prove.

The Three Foundational Invariants of LoopProof

1. Gated Environmental Impact (D-08): No carbon offset, landfill diversion tonnage, or ESG impact number is calculated, displayed in the UI, or exported via API until a batch reaches the `VERIFIED` terminal state. Unverified batches display explicit "Impact Unavailable" notices.

2. Enforced Independence (D-15, D-17): The verifier must be different from the assigned collector and processor. Self-verification or circular reviews trigger hard reverts.

3. Mass-Balance Conservation (D-05): The physical law of mass conservation is enforced at integer gram precision: total reported output mass ≤ recorded intake mass across all claimed fractions.

Human-in-the-Loop AI Classification (D-06, D-07)

Computer vision or LLM-based device classification models are treated strictly as *advisory suggestions*. A human registrar must review and confirm the device category code. No AI-controlled key holds any lifecycle role, and AI cannot approve batches or verify recycling claims.

Zero Tokenomics & Anti-Greenwashing (D-09)

LoopProof deliberately excludes speculative tokens, NFTs, carbon credit minting, and DAOs. By eliminating secondary financialization, the system removes perverse incentives to generate fictitious recycling records for token rewards.

The Truth vs. Trust Spectrum in Digital Product Passports

A cryptographic hash functions as an unforgeable digital notary stamp: it proves that an uploaded bill of lading, weighbridge ticket, or laboratory assay has not been modified by a single bit since anchoring. It does not certify that the physical scale was calibrated or that the operator was truthful. LoopProof addresses this reality by pairing immutable SHA-256 evidence-file integrity checking with institutional separation of duties, role-based liability, and mathematical mass-balance limits.

Physical Material Stream Complexities & Basel Convention Compliance

Printed Circuit Boards (PCBs):
High precious metal concentration (gold, copper, palladium) combined with hazardous brominated flame retardants (BFRs) and lead soldering. Without output mass-balance gating, valuable metals are stripped while toxic residues are dumped or informally burned.

IT & Telecom Hardware (Servers, Laptops):
Prone to illicit transboundary shipment under fraudulent "second-hand reuse donation" manifests, bypassing Basel Convention hazardous waste transport restrictions. LoopProof binds assigned facilities directly to certified custody logs.

System Comparison: Conventional Practice vs. LoopProof Architecture

DIMENSION	CONVENTIONAL E-WASTE TRACKING	LOOPPROOF ARCHITECTURE
Claim Timing	Impact booked at intake / drop-off aggregation.	Impact strictly gated until post-processing independent verification.
Evidence Integrity	Static unhashed PDF uploads, editable invoices, or self-assertions.	Cryptographic SHA-256 digests anchored permanently. Duplicate hashes rejected.
Role Governance	Aggregator or processor self-issues compliance certificate.	Rigid role separation: Registrar, Collector, Processor, and Verifier.
Mass Balance	Not enforced or calculated as coarse regional estimates.	Programmatic invariant: total reported output mass ≤ recorded intake mass.
Ledger Claim	Misleading claim that "blockchain proves recycling."	Truthful claim: proves who attested what, when, with which unaltered evidence.

3. Evidence-Gated Lifecycle State Machine

SPECIFICATION & RULES

Every e-waste batch in LoopProof progresses through an explicit, non-bypassable finite state machine. Transitions are executed through role-gated smart contract functions and PostgreSQL `SECURITY DEFINER` procedures. Arbitrary state regression is mathematically impossible; the only backward branch is a structured audit return requiring fresh evidence.

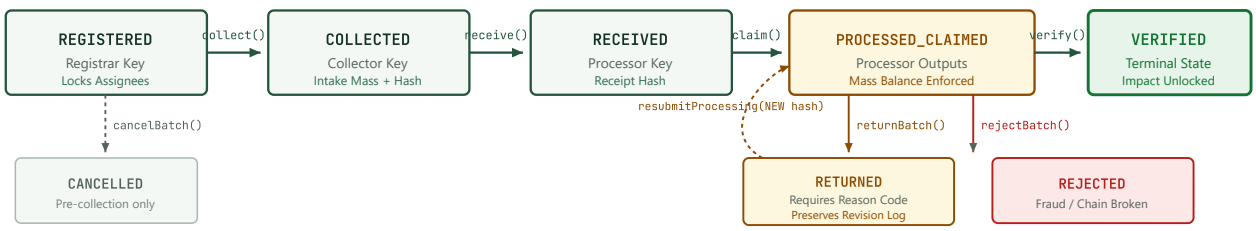


Figure 1: LoopProof State Machine Architecture with Role Assignments, Transition Gates, and Audit Feedback Loop.

Exhaustive State Machine Transition Specification

FROM STATE	FUNCTION (ACTION)	TO STATE	AUTHORIZED ACTOR	REQUIRED PAYLOAD & INVARIANTS
`NONE`	`registerBatch`	`REGISTERED`	`REGISTRAR_ROLE`	Unique claim key, category, collector, processor, verifier (distinct).
`REGISTERED`	`recordCollection`	`COLLECTED`	Assigned Collector	Intake mass > 0 (integer grams), new SHA-256 evidence hash.
`REGISTERED`	`cancelBatch`	`CANCELLED`	Assigned Registrar	Pre-collection window only. Terminal state.
`COLLECTED`	`recordReceipt`	`RECEIVED`	Assigned Processor	Facility intake confirmation, new SHA-256 evidence hash.
`RECEIVED`	`submitProcessing`	`PROCESSED_CLAIMED`	Assigned Processor	1–16 material fractions, total output mass ≤ intake mass, new SHA-256 hash.
`PROCESSED_CLAIMED`	`verifyBatch`	`VERIFIED`	Assigned Verifier	Checklist confirmation. Unlocks impact numbers. Terminal.
`PROCESSED_CLAIMED`	`returnBatch`	`RETURNED`	Assigned Verifier	Mandatory ReturnReason code (1..4). Non-terminal.
`PROCESSED_CLAIMED`	`rejectBatch`	`REJECTED`	Assigned Verifier	Mandatory RejectReason code (1..4). Terminal.
`RETURNED`	`resubmitProcessing`	`PROCESSED_CLAIMED`	Assigned Processor	Replaces output set, requires *genuinely fresh* SHA-256 evidence hash.

Revision History Tracking (DECISIONS D-13)

When a verifier returns a claim, the processor cannot erase the past. Resubmission replaces the active output set for review, but permanently preserves previous revisions (historical evidence hashes, timestamps, reported output masses, and return reasons) in an append-only audit trail queryable via `getRevision()`.

Global Evidence Hash Uniqueness (DECISIONS D-04)

The contract maintains a global evidence mapping: reusing a previously anchored SHA-256 digest—within the same batch or across any other batch in the system—reverts immediately. Resubmitting after a return requires genuinely fresh primary documentation.

Explicit Typed Reason Codes (DECISIONS D-14)

`RETURNREASON (NON-TERMINAL)`

- `INSUFFICIENT_EVIDENCE (1)` : Incomplete stage records.
- `EVIDENCE_UNREADABLE (2)` : Corrupted file or hash mismatch.
- `OUTPUT_DATA_INCOMPLETE (3)` : Missing fractions/materials.
- `MASS_DISCREPANCY (4)` : Unexplained mass drop vs intake.

`REJECTREASON (TERMINAL)`

- `EVIDENCE_INCONSISTENT (1)` : Contradictory assay data.
- `MASS_BALANCE_IMPLAUSIBLE (2)` : Unrealistic recovery yields.
- `PROCESSOR_NOT_QUALIFIED (3)` : Facility uncertified for stream.
- `CHAIN_OF_CUSTODY_BROKEN (4)` : Missing transit custody handover.

4. Dual Implementation Architecture

FULL-STACK DESIGN

LoopProof implements a dual-track architecture: a **live web application** backed by Supabase Auth and PostgreSQL Row Level Security for interactive evaluation, alongside a **Solidity smart contract reference specification** verified locally in Hardhat. The live application records actions in Supabase PostgreSQL; smart contracts are currently tested locally and are not deployed to a public blockchain.

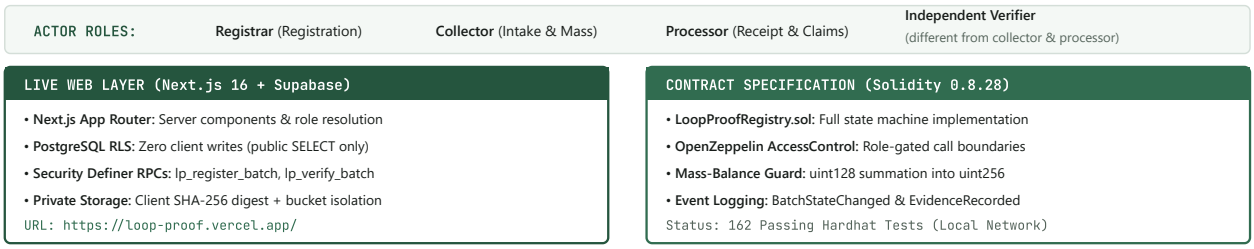


Figure 2: LoopProof Dual Architecture — Deployed Supabase Cloud Application & Solidity Smart Contract Reference Engine.

Database Architecture & Privilege Separation

TABLE / OBJECT	ACCESS CONTROL POLICY	DATA STRUCTURE & IMMUTABILITY GUARANTEE
`public.batches`	Authenticated SELECT only. Direct INSERT/UPDATE revoked.	Batch state, intake mass, locked assignees (collector, processor, verifier), timestamps.
`public.batch_history`	Authenticated SELECT only. Append-only via trigger.	State transition logs: `(from state, to_state, actor_id, evidence_hash, timestamp)`.
`public.evidence_records`	Authenticated SELECT only. Mutation revoked.	SHA-256 digest, storage path, MIME type, submitter role. Unique hash index.
`public.batch_outputs`	Authenticated SELECT only. Replaced per revision.	Output fractions (material code + mass in grams). Sum bounded by intake mass.
`public.batch_revisions`	Authenticated SELECT only. Append-only.	Snapshots of processing claims, historical hashes, and verifier return reason codes.

Security Definer RPC Execution Model

Direct write operations from web clients are blocked. All state transitions execute through transactional PostgreSQL `SECURITY DEFINER` functions (`lp_register_batch`, `lp_record_collection`, `lp_record_receipt`, `lp_submit_processing`, `lp_verify_batch`, `lp_return_batch`, `lp_reject_batch`, `lp_cancel_batch`). Each procedure enforces an explicit empty search path (`SET search_path = ''`) to eliminate search path injection vulnerabilities.

Privacy by Design & Zero-PII Policy

Supply chain transparency must not compromise enterprise confidentiality or donor privacy. Batches are indexed by a 32-byte salted hash commitment (`bytes32`). Underlying device serial numbers, IMEI tags, and customer identities remain off-chain. The `LoopProofRegistry.sol` ABI accepts zero `string` or dynamic `bytes` parameters, preventing accidental PII leakage on-chain.

Smart Contract Interface Specification (LoopProofRegistry.sol)

SOLIDITY FUNCTION	REQUIRED CALLER ROLE	STATE GUARD & INVARIANTS	EMITTED EVENT
`registerBatch()`	`REGISTRAR_ROLE`	Batch does not exist; claimKey unused; assignees valid & distinct.	`BatchRegistered`
`recordCollection()`	Assigned Collector	State is `REGISTERED`; intakeMass > 0; evidenceHash unused.	`BatchStateChanged`
`recordReceipt()`	Assigned Processor	State is `COLLECTED`; evidenceHash unused.	`BatchStateChanged`
`submitProcessing()`	Assigned Processor	State is `RECEIVED`; total output mass ≤ intake mass; 1..16 fractions; new hash.	`BatchStateChanged`
`verifyBatch()`	Assigned Verifier	State is `PROCESSED_CLAIMED`; verifier distinct from collector & processor.	`BatchVerified`
`returnBatch()`	Assigned Verifier	State is `PROCESSED_CLAIMED`; ReturnReason in 1..4.	`BatchStateChanged`
`rejectBatch()`	Assigned Verifier	State is `PROCESSED_CLAIMED`; RejectReason in 1..4.	`BatchStateChanged`
`cancelBatch()`	Assigned Registrar	State is `REGISTERED` (pre-collection only). Admin cannot cancel.	`BatchStateChanged`

5. Verification Results & Engineering Rigor

TESTED SCENARIOS

162
Solidity Mocha Tests Passing (Local Hardhat)

55
Web & Schema Guard Tests Passing (Node Runner)

64
Exhaustive State × Action Matrix Reversions

0
TypeScript Typecheck & Build Errors

Test Suite Coverage Breakdown

TEST SUITE CATEGORY	COUNT	VERIFIED INVARIANTS & BOUNDARY CONDITIONS
Access Control & Roles	11 tests	Only assigned accounts with active roles can execute transitions. Role revoking blocks pending actions.
Canonical Lifecycle Paths	11 tests	Linear progression `REGISTERED → COLLECTED → RECEIVED → PROCESSED → VERIFIED`. Multi-batch isolation.
Exhaustive State Matrix	64 tests	Parameterized 8 × 8 `(State × Action)` matrix: every unauthorized transition pair cleanly reverts.
Terminal Immutability	3 tests	`VERIFIED`, `REJECTED`, and `CANCELLED` batches reject all subsequent function calls.
Evidence & Key Rejection	10 tests	Duplicate claim keys and previously used evidence SHA-256 hashes revert globally across the system.
Verifier Independence	5 tests	Self-verification by collectors or processors holding `VERIFIER_ROLE` is strictly rejected.
Mass-Balance Validation	9 tests	Total output mass ≤ intake mass, intake = 0 revert, intake + 1g revert, bounds (1..16 fractions), overflow protection.
Revisions & Reason Codes	19 tests	Revision history preservation, fresh evidence requirement, typed ReturnReason and RejectReason enums.
Web App & Schema Guards	55 tests	Node test runner: route isolation, demo data invariants, enum alignment with Solidity, client RPC helpers, and static migration security checks.

6. Transparent Limitations & Disclosures

HONEST SCOPE

1. Synthetic Prototype Data: The 7 demo batches on the public dashboard (`web/lib/demo-data.ts`) are synthetic mock records created for UI demonstration. They do not represent real-world physical devices, recycling facilities, or actual laboratory assays.

2. Evidence-File Integrity vs. Physical Truth: An anchored SHA-256 digest checks evidence-file integrity—confirming that the digital document has not been altered since registration. It does *not* prove that the document was truthful when written, nor that physical recycling took place.

3. Local Hardhat Execution Status: Solidity contracts have been thoroughly tested on a simulated local Hardhat network (EVM `chainId 31337`). They are not deployed to any public testnet or mainnet. The live app records actions in Supabase, not on-chain.

4. Absence of Security Audit: The codebase is an academic prototype developed for the hackathon. It has not undergone an independent professional third-party smart contract security audit.

7. Future Work & Production Roadmap

NEXT MILESTONES

Phase 1: Public Testnet & Subgraph Indexing

Deploy `LoopProofRegistry.sol` to the Base Sepolia testnet with public Etherscan verification. Implement a decentralized indexing subgraph (The Graph / Goldsky) to serve historical batch revisions and queryable proof trees directly to client applications.

Phase 2: Multi-Party Verifier Quorum

Transition from single-verifier assignments to a decentralized verification pool. Implement threshold BLS signatures requiring k-of-n independent accredited auditors to sign off on high-volume or hazardous material streams.

Phase 3: Automated IoT Scale Integration

Connect certified industrial weighing platforms directly to registrar and collector modules. Use cryptographic hardware security modules (HSMs) on weighbridges to sign gross-tare-net tickets directly at the facility gate.

Phase 4: Downstream Material Passports

Issue ERC-1155 tokens or W3C Verifiable Credentials for recovered commodity fractions (copper, aluminum, refined gold) to enable circular manufacturing procurement with verifiable recycled content.

Student Builder Commentary & Hackathon Reflection

Our central engineering objective was resisting the temptation of Web3 speculation. Rather than minting tokens or claiming that distributed ledgers magically eliminate physical fraud, we constrained LoopProof to what cryptography can honestly guarantee: evidence-file integrity checking, rigorous separation of duties, and mathematical mass-balance conservation. This discipline produced a resilient prototype that directly addresses greenwashing in e-waste supply chains.

Project Evaluation Links: Explore the live interactive application at https://loop-proof.vercel.app/ or inspect the complete smart contracts, migrations, and test suites at https://github.com/vishu2212/LoopProof.